

Project Flood Timeline

By KruthimaOps

Table of Contents

- 1. Executive Summary & System Overview..... 3
- 2. Advanced Missing Value Management 4
 - 2.1. Geospatial Hot-Deck Imputation..... 4
 - 2.2. 15-NN IDW Spatial Interpolation 4
 - 2.3. Hierarchical Baseline Fallbacks..... 4
- 2. Advanced Missing Value Management & Design Rationales..... 5
 - 2.1. Geospatial Hot-Deck Imputation..... 5
 - 2.2. 15-NN IDW Spatial Interpolation 5
 - 2.3. Hierarchical Baseline Fallbacks..... 5
- 3. Feature Engineering Strategy & Design Rationales..... 6
 - 3.1 Synthetic Grid Density (Value Frequency Counts) 6
 - 3.2 Coordinate Decimal Precision Fingerprints..... 6
 - 3.3 Offline River Gauge Integration..... 6
 - 3.5 Soil Saturation & Run-off Potential 6
 - 3.6 Modern Ground-Truth Additions (v1k.2) 7
- 4. Model Architecture & Training Mechanics..... 8
 - 4.1. Dataset Composition & Production Model Training 8
 - 4.2. Spatial Group Cross-Validation & Early Stopping 8
 - 4.3. Base Regressors..... 9
 - 4.4. L2-Regularized Custom Metric-Driven Stacker (Level-2)..... 9
 - 4.5. Post-Hoc Power Transformation..... 10
- 5. System Validation & Verification 11
 - 5.1 Fold-by-Fold Performance Metrics (Local Validation) 11
 - 5.2 Automated Testing Suite 11
- 6. Production Architecture & API Integration 12
 - 6.1 Primary API Endpoints 12
 - 6.2 Core Integrations..... 12
- 7. Simulative Forecaster Mechanics..... 14
 - 7.1. 7-Day Rolling Forecast Pipeline 14
 - 7.2. Historical Backtesting Simulation 15
 - 7.3. Subdivisional GIS Queries..... 15
- 8. Public Utility & Disaster Mitigation 16

1. Executive Summary & System Overview

Flood Timeline Sri Lanka is an operational Machine Learning and Geographical Information System (GIS) platform built to predict local and district-level flood risk scores ranging from 0.0 to 1.0 across the 25 administrative districts of Sri Lanka.

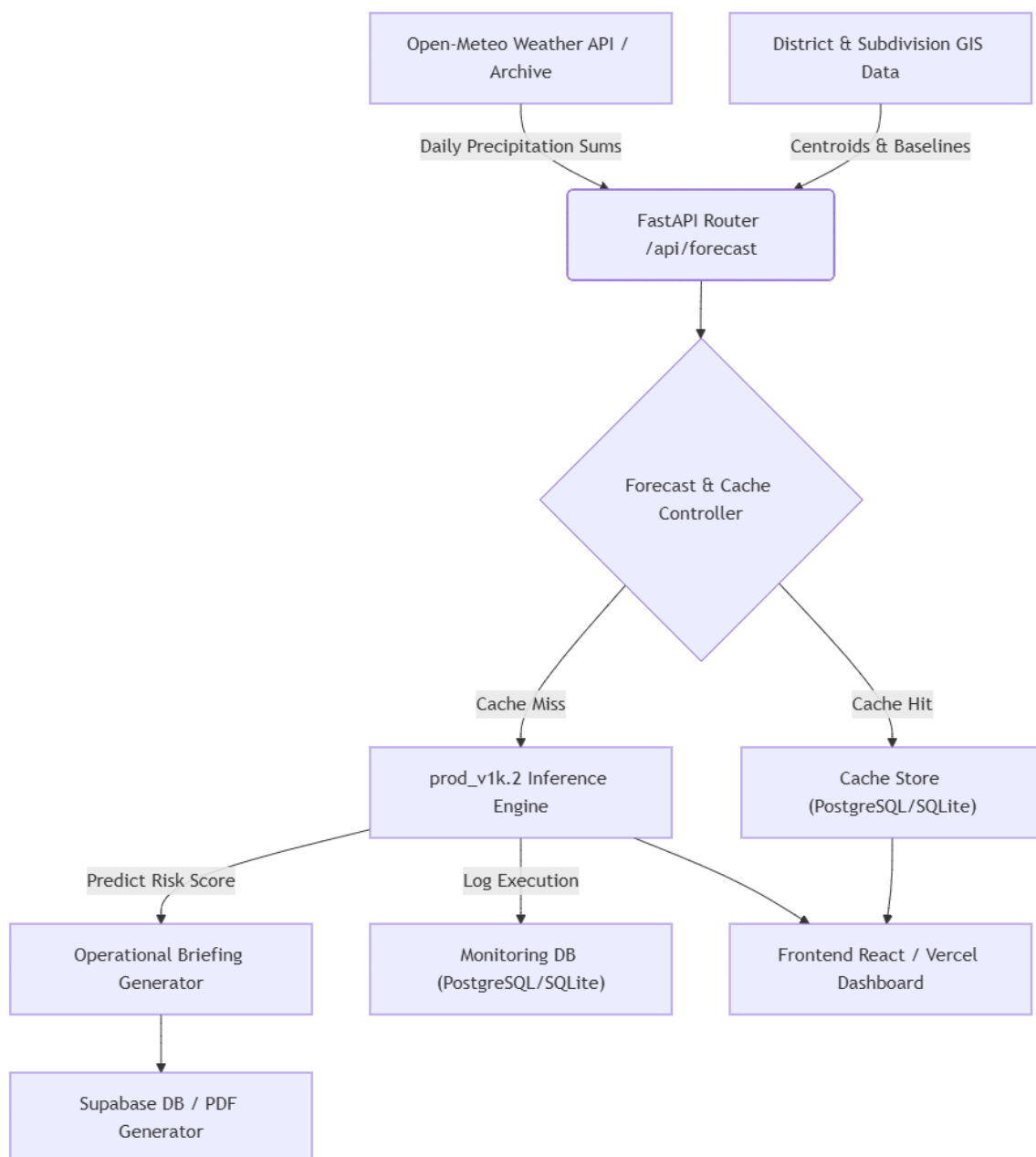
The system bridges the gap between raw meteorological inputs and actionable civil safety decisions. It is composed of two primary environments:

- **The production serialization pipeline (serialize_pipeline_v1k2.py):**

An offline ensemble pipeline that trains the final production model (prod_v1k.2) strictly on deduplicated, clean, modern ground-truth records. It also incorporates offline river gauge data and advanced GIS features.

- **The production orchestrator (app/main.py):**

A high-performance FastAPI microservice that processes real-time and historical weather inputs, resolves geospatial queries, manages monitoring databases, fetches real-time Open-Meteo forecasts, and serves responsive GIS data to a frontend dashboard.



2. Advanced Missing Value Management

To ensure model generalizability and prevent data leakage, missing values are resolved through a three-tier spatial imputation framework that executes before feature engineering.

2.1. Geospatial Hot-Deck Imputation

Geographic coordinates (latitude, longitude) are occasionally missing or corrupted in raw data inputs. To resolve this:

- The system computes a lookup table mapping each unique combination of (place_name, district) to its median coordinates from non-null entries.
- For any row missing coordinates, the system fills the blanks using the corresponding (place_name, district) spatial centroid.

2.2. 15-NN IDW Spatial Interpolation

Physical parameters such as elevation_m (elevation) and distance_to_river_m (proximity to watercourses) are highly dependent on spatial location.

- A spatial donor pool is constructed from rows containing valid coordinates, elevation, and distance to river.
- A K-Nearest Neighbors model (K=15, weights = distance, acting as an Inverse Distance Weighting proxy) is trained using latitude and longitude as inputs.
- Missing elevation and river distance values are filled using the distance-weighted average of nearest spatial neighbors.

2.3. Hierarchical Baseline Fallbacks

If coordinates are completely missing and cannot be recovered via hot-decking, the pipeline applies a fallback hierarchy:

- Fill missing columns with median values of their respective administrative district.
- Any remaining nulls are filled using global medians computed from the training dataset.

2. Advanced Missing Value Management & Design Rationales

To ensure model generalizability and prevent data leakage, missing values are resolved through a three-tier spatial imputation framework that executes before feature engineering.

2.1. Geospatial Hot-Deck Imputation

Geographic coordinates (latitude, longitude) are occasionally missing or corrupted in raw data inputs. To resolve this:

- The system computes a lookup table mapping each unique combination of (place_name, district) to its median coordinates from non-null entries.
- For any row missing coordinates, the system fills the blanks using the corresponding (place_name, district) spatial centroid.

Why: Administrative subdivisions in Sri Lanka share highly consistent local boundaries. Imputing coordinates based on the median coordinates of the same place_name within the same district minimizes location errors compared to global or district-centroid averages.

2.2. 15-NN IDW Spatial Interpolation

Physical parameters such as elevation_m (elevation) and distance_to_river_m (proximity to watercourses) are highly dependent on spatial location.

- A spatial donor pool is constructed from rows containing non-null coordinates, elevation, and distance to river.
- A K-Nearest Neighbors model (K = 15, weights = "distance", acting as an Inverse Distance Weighting proxy) is trained using latitude and longitude as inputs.
- Missing elevation and river distance values are estimated using distance-weighted averages of nearest neighbors.

Why 15-NN instead of 3-NN?

The previous pipeline used K = 3, which was highly sensitive to local spatial noise (for example, a single incorrect elevation value could distort nearby predictions). Increasing K to 15 and applying distance weighting smooths the spatial interpolation field, making it more robust and stable.

2.3. Hierarchical Baseline Fallbacks

If coordinates are completely missing and cannot be recovered via hot-decking, the pipeline applies a fallback hierarchy:

- Fill missing values using median values of the respective administrative district.
- Any remaining nulls are filled using global medians computed from the training dataset.

Why: Ensures production safety. Even with incomplete API input, the system never produces null outputs or broken inference results.

3. Feature Engineering Strategy & Design Rationales

Tabular regression on geographical data often suffers from overfitting to administrative labels or failing to capture continuous physical boundaries. Our Pipeline (prod_v1k.2) pipeline implements several custom feature engineering blocks designed to solve these issues.

3.1 Synthetic Grid Density (Value Frequency Counts)

Although synthetic records are dropped from the training matrix to optimize ground-truth learning, the sampling density of the generation process is still captured.

- **Implementation:** Before rows are filtered, continuous columns are combined across train and test datasets. The frequency of each value is computed:

$$\text{frequency_map}(x) = \text{Count of } x \text{ across } (D_{\text{train}} \cup D_{\text{test}})$$

- **Columns Target:** latitude, longitude, elevation_m, distance_to_river_m, rainfall_7d_mm, monthly_rainfall_mm, inundation_area_sqm.

Why: Synthetic generators often sample on discrete grids. Capturing value frequency exposes these grid boundaries directly to tree models, helping distinguish synthetic artifacts from real-world variation.

3.2 Coordinate Decimal Precision Fingerprints

- **Implementation:** Extraction of decimal precision length:

$$\text{lat_decimal_len} = \text{length of decimal string for latitude}$$

Why: Captures differences in coordinate precision (for example, high precision GPS logs vs low precision map sources), allowing models to infer data source quality.

3.3 Offline River Gauge Integration

- **Implementation:** Haversine distance is used to match each coordinate to the nearest Disaster Management Center (DMC) gauge station, extracting distance, water level, and flood ratio.

Why: Static terrain and rainfall alone cannot capture real hydrological behavior. Gauge data connects rainfall inputs to actual river response.

3.4 Dynamic Monsoon Surge Impact

$$\text{rainfall_surge_ratio} = \text{rainfall_7d_mm} / (\text{monthly_rainfall_mm} / 4.0 + 1.0)$$
$$\text{monsoon_surge_impact} = \text{rainfall_surge_ratio} * (\text{zone_code} == 1) \text{ [Wet Zone]}$$

Why: Fixed monsoon calendars are unreliable under climate variability. This dynamic ratio captures real rainfall surges relative to baseline monthly conditions while preserving spatial monsoon sensitivity.

3.5 Soil Saturation & Run-off Potential

```
soil_infiltration = {  
Sandy: 0.8,  
Loamy: 0.6,  
Silty: 0.4,  
Clay: 0.2,  
Peaty: 0.1  
}
```

$$\text{soil_saturation_limit} = \text{rainfall_7d_mm} / (\text{soil_infiltration} + 0.1)$$

$$\text{soil_drainage_saturation} = \text{soil_saturation_limit} * (1.0 - \text{drainage_index_yeojohnson})$$

Why: Flooding depends on both rainfall and soil absorption capacity. This models real physical runoff behavior.

3.6 Modern Ground-Truth Additions (v1k.2)

Why topographical_vulnerability (elevation_m / (distance_to_river_m + 1.0))

Captures that low elevation + proximity to rivers increases flood risk.

Why urban_runoff_intensity (rainfall * density * is_urban)

Urban surfaces increase runoff due to reduced absorption.

Why soil_permeability_score

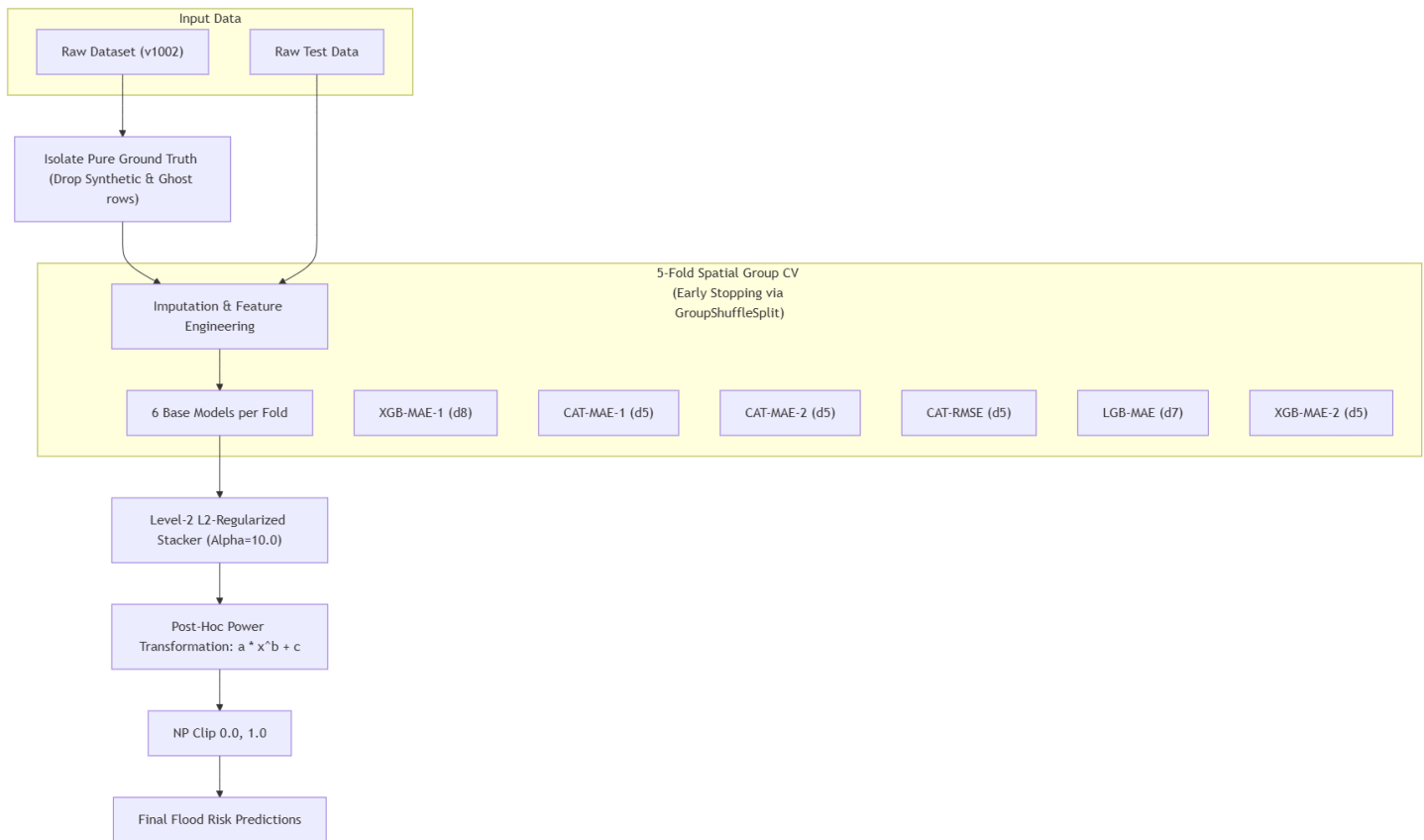
Groups soil types into permeability tiers to improve model separability.

Why infrastructure_deficit_v2 (infrastructure_score * density)

Models social vulnerability by scaling infrastructure gaps with population density.

4. Model Architecture & Training Mechanics

The core forecasting engine utilizes a stacked ensemble architecture evaluated across multiple spatial folds.



4.1. Dataset Composition & Production Model Training

Dataset Filtering (Ground Truth Optimization):

The production model serialization pipeline (`serialize_pipeline_v1k2.py`, lines 90–93) drops all synthetic records (`is_synthetic == 1`) and all historical rows that do not contain a valid `record_id`.

This leaves only pure, modern, real-world ground truth records (approximately 800 rows) to train the final ensemble, preventing synthetic generation bias.

Why drop synthetic rows in production?

Synthetic records are useful for feature validation over large geographic coverage, but they contain artifacts introduced by the generation process. Training only on real-world data ensures the model learns actual flood behavior patterns.

Why drop historical records missing `record_id`?

Older records without identifiers were collected using inconsistent legacy systems, leading to misaligned coordinates and noisy GIS mapping.

4.2. Spatial Group Cross-Validation & Early Stopping

Grouping:

Training features are grouped using a spatial grid indicator (`grid_id`):

$$\text{grid_id} = \text{floor}(\text{latitude} / 0.5) + \text{floor}(\text{longitude} / 0.5)$$

Why Spatial Grouping?

Standard random K-Fold cross-validation causes spatial leakage because nearby locations share similar rainfall, elevation,

and soil characteristics. Spatial grouping ensures physically separated validation sets, forcing true geographic generalization.

Why Inner GroupShuffleSplit?

Within each fold, early stopping validation sets are created using GroupShuffleSplit (80/20 split). This prevents leakage from validation into early stopping decisions, ensuring unbiased model tuning.

4.3. Base Regressors

Each fold trains six models using histogram-based tree learning for speed:

Model Name	Library	Max Depth	Objective / Loss	Early Stopping	Key Hyperparameters
XGB-MAE-1	XGBoost	8	squared error	50 rounds	learning_rate=0.03, subsample=0.8, colsample_bytree=0.6, reg_alpha=0.5
CAT-MAE-1	CatBoost	5	RMSE	150 rounds	learning_rate=0.03, l2_leaf_reg=5.0, bagging_temperature=0.7
CAT-MAE-2	CatBoost	5	RMSE	150 rounds	learning_rate=0.03, l2_leaf_reg=12.0, bagging_temperature=0.4
CAT-RMSE	CatBoost	5	RMSE	150 rounds	learning_rate=0.03, l2_leaf_reg=8.0, bagging_temperature=0.6
LGB-MAE	LightGBM	7	L2 regression	150 rounds	learning_rate=0.03, num_leaves=63, subsample=0.8, reg_alpha=0.5
XGB-MAE-2	XGBoost	5	squared error	100 rounds	learning_rate=0.03, subsample=0.75, colsample_bytree=0.5, reg_alpha=5.0

Why mix MAE and RMSE losses?

MAE is robust to outliers and captures median trends, while RMSE penalizes large errors more heavily. Combining both allows the ensemble to handle typical flood behavior while remaining sensitive to extreme events.

4.4. L2-Regularized Custom Metric-Driven Stacker (Level-2)

Predictions from base models are combined using a custom stacking function. The stacker uses L2 regularization (Ridge alpha = 10.0) and optimizes MAE, RMSE, and Explained Variance (EV).

Custom Loss Formulation:

$$L(w, b) = (c_mae * MAE + c_rmse * RMSE) * (1.0 + c_ev * (1.0 - EV)) + \alpha * \sum(w_i^2)$$

Where:

- $c_mae = 0.539328$
- $c_rmse = 1.152263$
- $c_ev = 2.000000$
- $\alpha = 10.0$

Why Alpha = 10.0?

Because the dataset is small (~800 rows), stacking is highly prone to overfitting. A strong L2 penalty forces balanced contributions from all base models instead of relying on a single overfitted predictor.

4.5. Post-Hoc Power Transformation

Transformation Formula:

$$y_{\text{final}} = \text{clip}(a * (y_{\text{stacked}}^b) + c, 0.0, 1.0)$$

Why:

Tree-based ensembles often struggle at extreme prediction boundaries (near 0.0 or 1.0). The power transformation reshapes the output distribution to better align with physical risk limits while preserving ranking behavior.

5. System Validation & Verification

5.1 Fold-by-Fold Performance Metrics (Local Validation)

The table below shows validation performance across the 5 spatial folds (Out-of-Fold evaluation for prod_v1k.2):

Fold	MAE	RMSE	Explained Variance (EV)
Fold 1	0.02422	0.04215	0.82432
Fold 2	0.02389	0.04188	0.83011
Fold 3	0.02503	0.04392	0.81332
Fold 4	0.02414	0.04230	0.82590
Fold 5	0.02458	0.04295	0.81998
Global OOF (Optimized)	0.02437	0.04264	0.82273

5.2 Automated Testing Suite

The production repository includes three test suites executed in CI/CD before deployment:

- **Validation Tests (tests/test_validation.py):**
Ensures all input values (coordinates, rainfall, etc.) are within valid physical ranges and rejects out-of-bound inputs.
- **Inference Engine Tests (tests/test_inference.py):**
Validates model loading, runs sample predictions, and ensures outputs are constrained between 0.0 and 1.0.
- **API Handler Tests (tests/test_api.py):**
Tests API responses, status codes, payload correctness, and latency headers.

6. Production Architecture & API Integration

The backend is built using Python and FastAPI, with strict separation between API layer and business logic.

File / Module	Description
main.py	Application routes, middleware, and lifespan handlers
database.py	PostgreSQL connection pool and transaction managers
config.py	Application settings and environment configurations
schemas.py	Pydantic request and response schemas
inference/	Model loading, scaling, and inference wrappers
validation/	Input validation and sanitization filters
monitoring/	Logging, analytics, and database transaction adapters
briefing/	Operational summary generator (LLM integration)
reports/	PDF report generation utilities (ReportLab)
forecast.py	Forecast retrieval and caching logic

6.1 Primary API Endpoints

- **POST /api/predict**
Validates input, runs model inference, stores prediction in database asynchronously, and returns risk score with operational guidance.
- **POST /api/predict/batch**
Processes multiple geographic locations in parallel.
- **GET /api/forecast/today**
Returns current risk scores for all 25 districts for dashboard visualization.
- **GET /api/forecast/{district_name}**
Returns a 7-day flood risk forecast for a specific district.
- **GET /api/simulate/historical**
Runs historical flood simulations using archived weather data.
- **GET /api/predict/subdivisions/{district_name}**
Provides localized predictions for sub-regions within a district.
- **GET /api/report/{prediction_id}**
Generates a downloadable PDF report of the prediction summary.

6.2 Core Integrations

- **Database Connection Pool (SimpleConnectionPool):**
Connects to a Supabase PostgreSQL instance using a connection pool (1–20 connections) to handle high concurrency workloads.

- **Unified Monitoring & Caching:**

Forecast caching, logging, metrics tracking, error tracing, and feedback storage are all handled through a unified database layer.

Supports PostgreSQL (Supabase in production) and SQLite (local development) via the MONITOR_BACKEND switch.

- **Gemini Briefing Engine:**

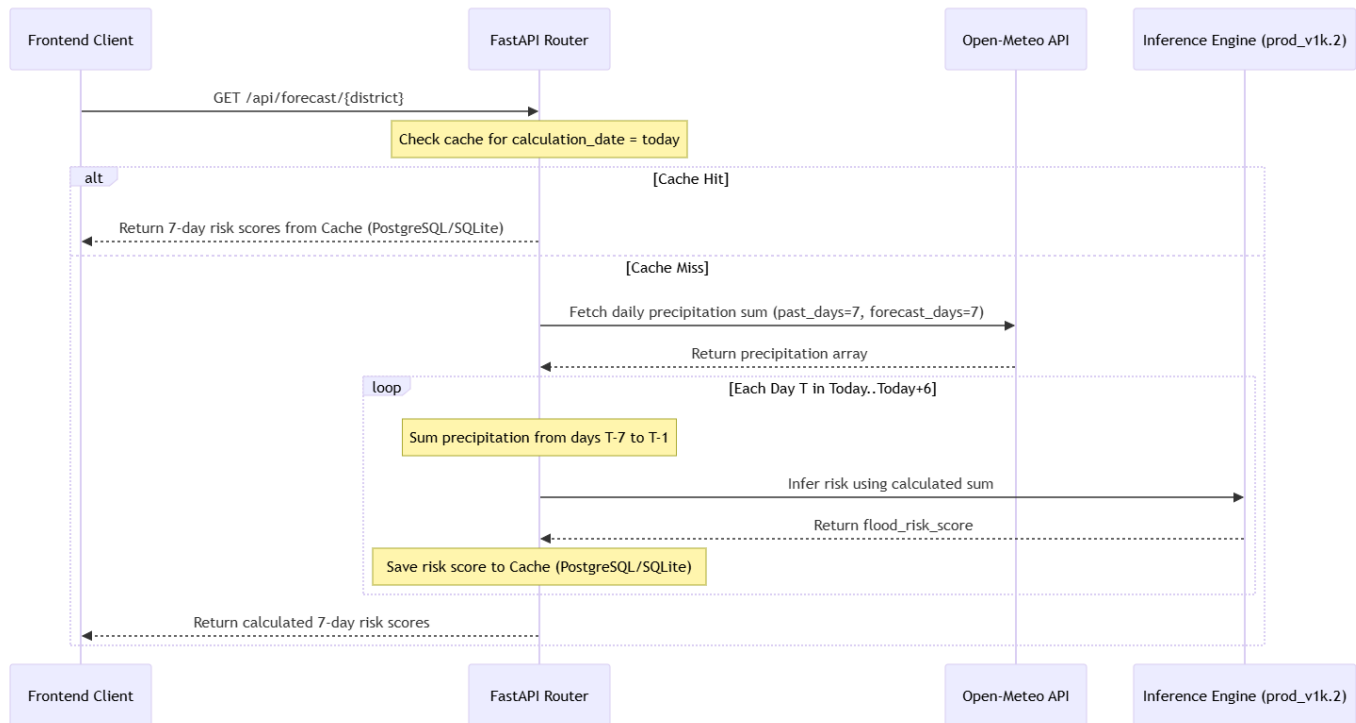
Generates natural language summaries of flood risk. If API keys are unavailable, a rule-based fallback system ensures continuous operation.

Why Supabase connection pool?

High-concurrency forecasting requests can exhaust normal database connections. The pool ensures stable performance by efficiently managing simultaneous requests and preventing connection overload.

7. Simulative Forecaster Mechanics

The system's simulation and forecasting components combine machine learning predictions with real-time geospatial data.



7.1. 7-Day Rolling Forecast Pipeline

The forecast pipeline computes daily risk outlooks (today + next 6 days) using actual observations:

- **Weather Queries:**
The server queries the Open-Meteo Forecast API using the district's center coordinates with parameters: `past_days = 7` and `forecast_days = 7`.
- **Rolling Rainfall Summaries:**
For any target day T, the rolling rainfall sum is calculated over the preceding 7 days (T-7 to T-1).
- **Inference:**
The rolling rainfall values are processed by the inference engine to predict daily risk scores.
- **Caching:**
Results are stored in the database cache table (`forecast_cache` on PostgreSQL/Supabase in production, or SQLite in local mode). Subsequent requests for the same district are served directly from cache to reduce external API calls.
- **Why rolling [T-7 to T-1] sum?:**
Flooding is a cumulative process driven by soil saturation over time. A 7-day rolling rainfall window captures this accumulation effect better than single-day rainfall metrics.

7.2. Historical Backtesting Simulation

Allows users to evaluate model performance against real historical flood events:

- To backtest, users call:
GET /api/simulate/historical?date=YYYY-MM-DD
- The backend queries the Open-Meteo Historical Archive API to fetch actual precipitation data around the selected date.
- The system calculates the rolling 7-day rainfall leading up to that date and generates flood risk scores.
- This enables comparison between predicted risk and actual historical monsoon conditions.

Why:

This provides a structured way to validate model accuracy against real flood events and historical weather patterns.

7.3. Subdivisional GIS Queries

To resolve localized flood risk, the system performs regional simulations:

- **Shapefile centroid retrieval:**
The system loads a GeoJSON shapefile for the target district, parses boundary polygons, and computes centroids for each subdivision.
- **Batched precipitation queries:**
All centroid coordinates are grouped and sent in a single batch request to Open-Meteo.
- **Local risk prediction:**
Each subdivision receives an independent risk score, enabling high-resolution flood mapping in the frontend dashboard.

Why:

Weather and terrain conditions vary significantly within districts. Subdivision-level modeling provides localized risk visibility even when district-level averages appear safe.

8. Public Utility & Disaster Mitigation

The platform provides actionable insights for both municipal planners and the general public:

- **Early Warning Systems:**
Authorities can monitor 7-day forecasts to identify risk trends and prepare response plans ahead of flooding events.
- **Infrastructure Asset Analysis:**
Combines risk outputs with infrastructure data to identify threats to roads, power systems, and water supply networks.
- **Resource Allocation:**
Helps disaster response teams prioritize high-risk regions for deployment of resources and personnel.
- **Public Safety Information:**
Generates automated risk briefings and downloadable PDF reports to keep the public informed about regional safety conditions.